Software Performance Engineering



Recitation 2.5

Atharva Kirkole

Thursday, Nov 5, 2026

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Some due dates

- Project 2 Beta submission + write-up: Friday, November 6
- Project 2 Final submission + write-up: Sunday, November 15
  - both moved back 3 days for the telerun outage
- Homework 5 (theory, individual): Friday, November 13
  - warm-up for Quiz 2 - treat it like a take-home quiz
- Telerun is back up. Re-run anything that failed last week.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

PROJECT 2 FINAL



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Basic collision check

```
for (int i = 0; i < n_spheres; i++)
  for (int j = i + 1; j < n_spheres; j++)
    if (check_for_collision(spheres, i, j, &minTime)) {
      indexCollider1 = i;  indexCollider2 = j; }
```

- Every check_for_collision() call computes:
  - Euclidean distance between two spheres (has a sqrt)
  - relative velocity of the two spheres
  - whether that relative motion closes the gap (another sqrt)
- Two square roots, for every pair, every ministep. Expensive.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

How can we improve this?

- For each sphere:
  - find where it ends up at the end of this timestep
  - use its start and end position to build a bounding box of
  - everywhere it could possibly be during the step
- Before any collision check:
  - do the boxes of the two spheres overlap?
  - that is 3 coordinate comparisons - no square roots
- The filter gives false positives but NEVER false negatives.
  - so a survivor still goes through the real check_for_collision()

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

How to further improve this?

- The box test fails fast: if the boxes miss on ONE axis,
  - they cannot collide, and y and z never need looking at.
- So do not even store 3 axes. Filter on ONE axis first,
  - and only look at the others for whatever survives.
- That one-axis filter is what makes a sort worth doing.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

The Sweep & Prune algorithm

- Three parts:
  - SORT - sort the boxes by their low edge on one axis (say x)
  - SWEEP - walk the sorted list; for each box, only compare
  - against boxes that start before this one ends
  - PRUNE - survivors may be false positives, so run the real
  - check_for_collision() on those, and only those

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Why is it faster?

- Worst case is unchanged - every box can overlap every other box.
- But in this project the spheres are mostly well scattered, so the
  - sweep throws away the overwhelming majority of pairs before
  - any square root is computed.
- You are not making check_for_collision() cheaper.
  - You are calling it far fewer times. That is the whole idea.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

HINTS FOR THE FINAL



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Hint 1: what render() actually does per pixel

- resolution^2 pixels, and for each one a loop over every sphere
  - until the first hit.
- At 320x320 with 800 spheres that is up to 81 million
  - ray-sphere intersection tests for ONE frame.
- Most of those tests are for spheres nowhere near that pixel.
- **Hint: a sphere only covers a small rectangle of the screen.**
  - Work out that rectangle once per sphere per frame, and a
  - pixel never tests a sphere that cannot cover it.
  - Recitation 2.1 already showed you how to project a box.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Hint 2: what is independent here?

- In render(), one pixel's colour depends on:
  - the sorted sphere list (read-only)
  - the spec (read-only)
  - and nothing else
- Two pixels never write the same memory. That is a cilk_for.
- In simulate(), update_accel_sphere(i) writes exactly one slot
  - and reads the first n_spheres entries.
- Watch for variables declared OUTSIDE the loop you parallelize -
  - a shared ray_t is a race. Declare it inside.
- Run make CILKSAN=1 before you believe any of this.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers